EEUC201 • ANALOG AND DIGITAL ELECTRONICS

Digital Logic Families and Design

Boolean Algebra • Logic Gates • Minimization Techniques • IC Logic Families

Course Module Presentation

ROADMAP

What We Will Cover

1. Boolean Algebra Basics
2. Basic Logic Gates & Truth Tables
3. Universal Gates
4. Laws of Boolean Algebra
5. De Morgan's Theorem
6. Min Terms, Max Terms, SOP & POS
7. Karnaugh Maps & Minimization
8. Don't Care Conditions
9. Introduction to Logic Families
10. RTL, DCTL, IIL, DTL
11. TTL Logic
12. CMOS Logic & Family Comparison

Foundations of Digital Logic

Boolean Algebra is a mathematical system used to analyze and design digital circuits. It deals with variables that take only two values — **0 (False/Low)** and **1 (True/High)**.

- Introduced by George Boole (1854)
- Basic operators: AND (.), OR (+), NOT (')
- Used to represent, simplify & implement logic circuits
- Forms the theoretical basis of digital system design

BINARY LOGIC

1 — TRUE / HIGH / ON

0 — FALSE / LOW / OFF

Three fundamental operations combine these binary values to build every digital function:

AND (·) | OR (+) | NOT ($\bar{}$)

Basic Logic Gates I — AND, OR, NOT

A·B

AND Gate

Output is HIGH only when all inputs are HIGH.

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

A+B

OR Gate

Output is HIGH when at least one input is HIGH.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

A'

NOT Gate

Output is the inverse (complement) of the input.

A	Y
0	1
1	0

Basic Logic Gates II — NAND, NOR, XOR, XNOR

(A·B)'

NAND

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

(A+B)'

NOR

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

A⊕B

XOR

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

(A⊕B)'

XNOR

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

NAND and NOR are called **Universal Gates** — any Boolean function can be implemented using only NAND gates, or only NOR gates.

Universal Gates — NAND & NOR

A gate is called “universal” if every basic gate (NOT, AND, OR) can be built from it alone. NAND and NOR are the two universal gates used to implement any digital circuit.

NOT using NAND

Tie both inputs of a NAND gate together: Y = (A·A)' = A'

AND using NAND

A NAND gate followed by a NAND-based NOT gate: Y = ((A·B)')' = A·B

OR using NAND

Invert A and B first, then NAND them: Y = A'·B''... = A+B (De Morgan)

Similarly, NOT, AND and OR gates can all be constructed using NOR gates alone — making both NAND and NOR sufficient building blocks for any combinational circuit.

Why it matters

Manufacturing only one type of gate simplifies IC fabrication, reduces cost, and standardizes digital circuit design.

Laws of Boolean Algebra

Law	OR form	AND form
Commutative	$A + B = B + A$	$A \cdot B = B \cdot A$
Associative	$(A+B)+C = A+(B+C)$	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$
Distributive	$A \cdot (B+C) = A \cdot B + A \cdot C$	$A+(B \cdot C) = (A+B) \cdot (A+C)$
Identity	$A + 0 = A$	$A \cdot 1 = A$
Null (Domination)	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotent	$A + A = A$	$A \cdot A = A$
Complement	$A + A' = 1$	$A \cdot A' = 0$
Absorption	$A + A \cdot B = A$	$A \cdot (A+B) = A$
Involution	$(A')' = A$	—

De Morgan's Theorem

Theorem 1

$$(A + B)' = A' \cdot B'$$

The complement of a sum equals the product of the complements.

Theorem 2

$$(A \cdot B)' = A' + B'$$

The complement of a product equals the sum of the complements.

PRACTICAL USE

- Converts AND-OR expressions into NAND / NOR form for implementation with universal gates
- Simplifies complemented Boolean expressions before minimization
- Bridges SOP and POS representations of a logic function

Min Terms and Max Terms

MIN TERM

A product (AND) term that contains every variable of the function, each appearing exactly once — in true or complemented form. It represents exactly one row of the truth table where the output is 1.

A	B	Minterm	Symbol
0	0	A'B'	m0
0	1	A'B	m1
1	0	AB'	m2
1	1	AB	m3

MAX TERM

A sum (OR) term that contains every variable of the function, each appearing exactly once. It represents exactly one row of the truth table where the output is 0.

A	B	Maxterm	Symbol
0	0	A+B	M0
0	1	A+B'	M1
1	0	A'+B	M2
1	1	A'+B'	M3

Sum of Products (SOP) & Product of Sums (POS)

SUM OF PRODUCTS — SOP

An OR of AND (minterm) terms — a canonical SOP expression is the sum of all minterms for which the function equals 1.

$$F(A,B,C) = \Sigma m(1,3,5,6,7)$$

Example: F = A'B'C + A'BC + AB'C + ABC' + ABC

Also called the “canonical/standard SOP” or minterm expansion. Directly implemented with AND gates feeding an OR gate.

PRODUCT OF SUMS — POS

An AND of OR (maxterm) terms — a canonical POS expression is the product of all maxterms for which the function equals 0.

$$F(A,B,C) = \Pi M(0,2,4)$$

Example: F = (A+B+C)(A+B'+C)(A'+B+C)

Also called the “canonical/standard POS” or maxterm expansion. Directly implemented with OR gates feeding an AND gate.

Karnaugh Maps (K-Maps)

A Karnaugh Map is a graphical tool that arranges truth-table outputs in a grid so that adjacent cells differ by only one bit (Gray-code order) — making it easy to spot and group terms for simplification.

- 2, 3, 4 (or more) variable maps supported
- Groups of 1's simplify SOP expressions
- Groups of 0's simplify POS expressions
- Groups must be sized in powers of 2 (1, 2, 4, 8 …)
- Larger groups → simpler resulting terms

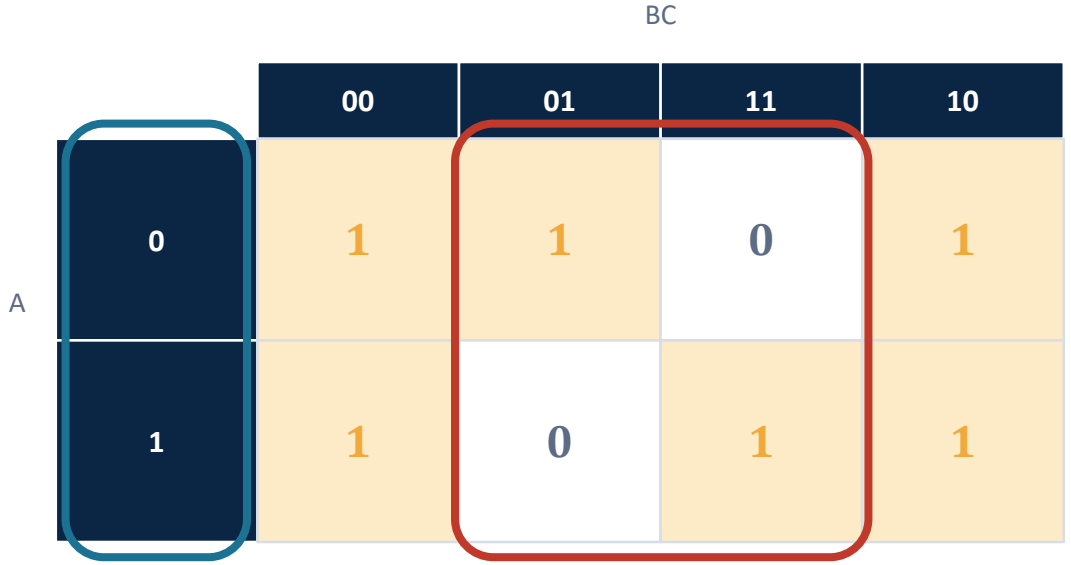
3-Variable K-Map — F(A,B,C)

A \ BC	00	01	11	10
0	0	1	1	0
1	0	1	1	0

Group = 4 cells → term "B"

Minimization of Logical Functions — Worked Example

F(A,B,C) = Σm(0, 1, 2, 5, 6, 7)



Steps

- Plot every minterm (1) at its Gray-coded A,BC position
- Group adjacent 1's into the largest possible power-of-2 blocks
- Teal group (B'C'): B=0, C=0 → term B'C'
- Red group (B+C column pair): B=1 or C=1 region → term C + B (simplified)
- OR all group terms together for the minimized expression

Minimized: F = B'C' + A

(illustrative grouping shown above; actual grouping choice depends on adjacency rules)

Don't Care Conditions

What is a Don't Care?

For certain input combinations, the output value never occurs in practice or is irrelevant to circuit operation. These are marked ‘X’ (or ‘d’) in the truth table / K-map instead of 0 or 1.

- Common in BCD, excess-3, and other restricted-input codes
- Treated as EITHER 0 or 1 — whichever gives a larger, simpler K-map group
- Using don't cares can further reduce the number of literals/gates
- Denoted Σd(...) alongside the minterm list

EXAMPLE — BCD Digit Detector

F(A,B,C,D) = Σm(1,3,5,7,9) + Σd(10,11,12,13,14,15)

BCD codes 10–15 never occur as valid inputs, so they are don't-cares. Allowing the map to treat them as 1 wherever convenient simplifies the final expression to:

F = D (detects odd digits)

Without don't cares, the expression would need many more literals to exclude codes 10–15 explicitly.

PART TWO

Logic Families and Design

RTL • DCTL • IIL • DTL • TTL • CMOS

Introduction to IC Logic Families

A logic family is a group of IC technologies built with compatible input/output structures, so gates from the same family can be interconnected directly. Each family trades off speed, power, cost, and noise immunity differently.

Fan-in / Fan-out

Number of inputs a gate can accept / outputs it can safely drive

Propagation Delay

Time taken for output to respond to an input change (speed)

Power Dissipation

Power consumed by a gate during operation

Noise Margin

Tolerance to unwanted voltage fluctuations without logic error

Fig. of Merit (Speed-Power Product)

Delay × Power — lower is better

Packing Density

Number of gates that can be fabricated per chip area

Resistor Transistor Logic (RTL)

One of the earliest IC logic families. Logic gates are built using resistors for input combination and bipolar transistors for switching / inversion — a basic NOR gate is the natural RTL structure.

- Structure: resistors combine inputs to the base of a switching transistor
- Very low fan-out (typically 4–5) due to resistive loading
- Low noise margin and relatively slow switching speed
- Largely obsolete today — replaced by faster, lower-power families

Direct Coupled Transistor Logic (DCTL)

An evolution of RTL where the base resistors are removed entirely — transistor bases are connected directly to the outputs of the driving gates, simplifying fabrication.

- No base resistors → higher packing density than RTL
- Susceptible to ‘current hogging’ — one transistor can starve others of base current
- Requires closely matched transistor characteristics, hard to guarantee in production
- Poor noise immunity limited its practical adoption

Integrated Injection Logic (IIL / I^2L)

A high-density bipolar logic family that replaces resistors with a constant-current ‘injector’ transistor, using multi-collector transistors to realize logic functions in very little chip area.

- Extremely high packing density — suited to LSI/VLSI bipolar designs
- Low power dissipation per gate at low switching speed
- Uses multi-collector NPN transistors driven by a PNP current injector
- Historically used in early digital watches, calculators, and telecom ICs

Diode Transistor Logic (DTL)

Combines diode logic gates (for AND/OR) with a bipolar transistor output stage acting as an inverting amplifier/switch — a direct predecessor to TTL.

- Input diodes perform the AND function; output transistor provides inversion and gain
- Better fan-out and noise margin than RTL/DCTL
- Slower than TTL due to diode/transistor switching delays
- Historically important as the design basis that TTL evolved from

Transistor Transistor Logic (TTL)

Replaces DTL's input diodes with a multi-emitter transistor, giving faster switching. TTL became the dominant standard logic family (74-series) for decades.

- Multi-emitter input transistor performs the AND function directly
- Totem-pole output stage gives fast, low-impedance switching
- Sub-families: Standard, Low-Power (L), Schottky (S), Low-Power Schottky (LS), Advanced (ALS/AS), Fast (F)
- Higher power dissipation than CMOS; supply typically 5V ± 5%

Complementary MOS Logic (CMOS)

Uses complementary pairs of PMOS and NMOS transistors so that, in steady state, one transistor is always OFF — giving near-zero static power dissipation.

- Extremely low static power consumption; power scales with switching frequency
- Very high noise margin and wide supply voltage range (3V–15V typical for 4000-series)
- High input impedance — sensitive to electrostatic discharge (ESD), needs protection diodes
- Dominant technology today: 74HC/HCT series, and virtually all modern VLSI/microprocessor design

Comparing the Logic Families

Family	Speed	Power	Fan-out	Noise Margin	Density
RTL	Slow	Medium	Low (4–5)	Low	Low
DCTL	Slow	Low	Low	Very Low	Medium
IIL	Slow–Med	Very Low	Medium	Medium	Very High
DTL	Medium	Medium	Medium	Medium	Medium
TTL	Fast	Medium–High	High (10)	Good	Medium
CMOS	Fast (freq.-dep.)	Very Low (static)	High	Very Good	Very High

CMOS dominates modern digital design due to its very low static power and high noise immunity; TTL remains common in legacy and interfacing applications.

SUMMARY

Key Takeaways

- Boolean algebra and its laws (incl. De Morgan's) provide the mathematical foundation for simplifying digital logic.
- SOP/POS, minterms/maxterms, and K-maps (with don't cares) are the standard tools for logic minimization.
- NAND and NOR are universal gates capable of implementing any Boolean function.
- Logic families (RTL → DCTL → IIL → DTL → TTL → CMOS) trace the evolution toward faster, lower-power IC design; CMOS is the modern standard.

Thank You